

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

Duration :60 Mins
On class
Total Marks:50

QUESTIONS:

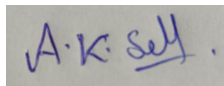
Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I A K Selva Prabhu(192421224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Compiler Design Assessment Tool 2

- Answers

Q1 (a) Identify all errors

Line	Error	Type	Reason
35	Missing ";"	Syntax	Missing semicolon.
37	findLength()	Semantic	Argument 'name' not passed.
38	if(count > 5	Syntax	Missing ')'
43	marks[4]	Runtime	Array index out of bounds.
50	case 2	Syntax	Missing ':'
58	*ptr	Runtime	Uninitialized pointer.
61	total/students	Runtime	Division by zero.
63-66	while loop	Logical	No i++ causing infinite loop.

Q1 (b) Classification

Error	Category
Missing semicolon	Syntax
Missing function argument	Semantic
Missing)	Syntax
Array out of bounds	Runtime
Missing :	Syntax
Uninitialized pointer	Runtime
Division by zero	Runtime
Infinite loop	Logical
Lexical Error	None

Q1 (c) Compilation phase

Category	Detected During	Explanation
Lexical	Lexical Analysis	Invalid tokens
Syntax	Syntax Analysis	Grammar checking
Semantic	Semantic Analysis	Meaning/function checking
Logical	Not detected by compiler	Wrong logic
Runtime	Execution	Occurs while running

Q1 (d) Corrected Program

```
main.c  [Icons]  Run  Output  Clear

1 #include <stdio.h>
2 #include <string.h>
3
4 int findLength(char str[]){ return strlen(str); }
5
6 int main(){
7     char name[20]="Compiler";
8     int count=findLength(name);
9     if(count>5) printf("Length is greater than 5\n");
10    int marks[4]={85,90,75,80};
11    printf("%d\n",marks[3]);
12    int choice=2;
13    switch(choice){
14        case 1: printf("Option 1\n"); break;
15        case 2: printf("Option 2\n"); break;
16        default: printf("Invalid Choice\n");
17    }
18    int value=100,*ptr=&value;
19    printf("%d\n",*ptr);
```

Length is greater than 5
80
Option 2
100
Average=20.00
1
2
3
4
5
=== Code Execution Successful ===

```
#include <stdio.h>
#include <string.h>

int findLength(char str[]){ return strlen(str); }

int main() {
    char name[20]="Compiler";
    int count=findLength(name);
    if(count>5) printf("Length is greater than 5\n");
    int marks[4]={85,90,75,80};
    printf("%d\n",marks[3]);
    int choice=2;
    switch(choice) {
        case 1: printf("Option 1\n"); break;
        case 2: printf("Option 2\n"); break;
        default: printf("Invalid Choice\n");
    }
    int value=100,*ptr=&value;
    printf("%d\n",*ptr);
    int total=100,students=5;
    float average=(float)total/students;
    printf("Average=%.2f\n",average);
    int i=1;
    while(i<=5){ printf("%d\n",i); i++; }
    return 0;
}
```

}

Q1 (e) Compiler Features

- Compiler warnings (-Wall, -Wextra)
- Static code analysis
- Type and semantic checking

Q2 (a) Syntax Errors

Line	Reason
88	Missing ')' in function definition
103	Missing ';' after printf
105	Missing ':' after case 90
115	Incorrect do-while structure
117	Missing ')' and ';' in printf
118	Misplaced closing brace

Q2 (b) Categories

Error	Category
Missing)	Parenthesis
Missing ;	Delimiter
Missing :	Switch-case
Incorrect do-while	Loop structure
Missing) and ;	Function statement
Misplaced }	Block structure

Q2 (c) Corrected Program

main.c

Share

Run

```
1 #include <stdio.h>
2
3 int maximum(int x,int y){
4     if(x>y) return x;
5     else return y;
6 }
7
8 int main(){
9     int marks[5]={70,80,90,85,95};
10    int max=maximum(marks[0],marks[1]);
11    switch(max){
12        case 80: printf("Maximum is 80\n"); break;
13        case 90: printf("Maximum is 90\n"); break;
14        default: printf("Other Value\n");
15    }
16    do{
17        printf("%d\n",max);
18        max--;
19    }while(max>75):
```

Output

Maximum is 80
80
79
78
77
76
Limit Reached

=== Code Execution Successful ===

```

#include <stdio.h>

int maximum(int x,int y){
    if(x>y) return x;
    else return y;
}

int main(){
    int marks[5]={70,80,90,85,95};
    int max=maximum(marks[0],marks[1]);
    switch(max){
        case 80: printf("Maximum is 80\n"); break;
        case 90: printf("Maximum is 90\n"); break;
        default: printf("Other Value\n");
    }
    do{
        printf("%d\n",max);
        max--;
    }while(max>75);

    if(max==75) printf("Limit Reached\n");
    return 0;
}

```

Q2 (d) Parser

The parser checks whether the program follows C grammar rules. It reports syntax errors such as missing semicolons, parentheses, or incorrect loop structures. Error recovery allows the compiler to continue parsing and report multiple errors.

Q2 (e) Improvements

- Clear error messages with line numbers
- Suggest possible corrections

Q3 (a) Errors

Line	Type	Reason
147	Lexical	Invalid '@' in variable name
148	Syntax	Missing semicolon
150	Semantic	Missing function argument
161	Syntax	Missing ':' after case
168	Runtime	Array out of bounds

170	Runtime	Uninitialized pointer
173	Runtime	Division by zero
174	Optimization	Unnecessary +0
176-179	Logical	Infinite loop

Q3 (b) Compiler Phase

Error	Phase
Invalid variable	Lexical
Missing ;	Syntax
Function argument	Semantic
Missing :	Syntax
result+0	Optimization
Array out of bounds	Runtime
Pointer	Runtime
Division by zero	Runtime
Infinite loop	Runtime/Logical

Q3 (c) Explanation

Lexical analysis detects invalid tokens. Syntax analysis detects grammar errors. Semantic analysis checks function calls and meanings. Intermediate code generation converts the corrected program into an intermediate form. Code optimization removes unnecessary operations. Runtime errors such as division by zero, invalid memory access, and infinite loops are detected only during execution.

Q3 (d) Corrected Program

main.c

Share

Run

Output

```

1 #include <stdio.h>
2
3 int factorial(int n){
4     if(n<=1) return 1;
5     return n*factorial(n-1);
6 }
7
8 int main(){
9     int num=5,value=10;
10    int result=factorial(num);
11    if(result>=100) printf("Large Number\n");
12    switch(result){
13        case 24: printf("Factorial of 4\n"); break;
14        case 120: printf("Factorial of 5\n"); break;
15        default: printf("Other Value\n");
16    }
17    int numbers[5]={2,4,6,8,10};
18    printf("%d\n",numbers[4]);
19    int data=50.*ptr=&data;
```

Large Number

Factorial of 5

10

50

Division=10

Sum=130

1

2

3

4

5

=== Code Execution Successful ===

```

#include <stdio.h>

int factorial(int n){
    if(n<=1) return 1;
    return n*factorial(n-1);
}

int main(){
    int num=5,value=10;
    int result=factorial(num);
    if(result>=100) printf("Large Number\n");
    switch(result){
        case 24: printf("Factorial of 4\n"); break;
        case 120: printf("Factorial of 5\n"); break;
        default: printf("Other Value\n");
    }
    int numbers[5]={2,4,6,8,10};
    printf("%d\n",numbers[4]);
    int data=50,*ptr=&data;
    printf("%d\n",*ptr);
    int a=50,b=5,c=a/b;
    printf("Division=%d\n",c);
    int sum=result+value;
    printf("Sum=%d\n",sum);
    int i=1;
    while(i<=5){ printf("%d\n",i); i++; }
    return 0;
}

```

Q3 (e) Improvements

- Advanced static code analysis
- Improved optimization and warning system